

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC PHENIKAA



BÁO CÁO DỰ ÁN CUỐI KÌ
CHỦ ĐỀ : HỆ THỐNG QUẢN LÝ VÀ ĐÁNH GIÁ
ĐỊA ĐIỂM DU LỊCH

Giảng viên : Ths. Phạm Đức Bắc

Học phần : Lập trình web ứng dụng trong lịch 2-1-3-25(N01)

Họ và tên sinh viên :

Nguyễn Thế Anh MSSV : 23016080

Nguyễn Mạnh Cường MSSV : 23016117

Vũ Hùng Hiến MSSV : 23016291

Hà Nội, tháng 5 năm 2026

Mục lục

1	Giới thiệu đề tài	3
1.1	Đặt vấn đề và Lý do lựa chọn đề tài	3
1.2	Mục tiêu phát triển hệ thống	4
1.2.1	Mục tiêu tổng quát	4
1.2.2	Mục tiêu cụ thể (Bám sát kiến trúc mã nguồn)	4
1.3	Đối tượng sử dụng và Phân quyền hệ thống	5
1.3.1	Khách vãng lai (Guest / Anonymous User)	5
1.3.2	Thành viên hệ thống (Authenticated User)	5
1.3.3	Quản trị viên tối cao (System Administrator)	6
1.4	Phạm vi nghiên cứu và Giải pháp công nghệ	6
1.4.1	Phạm vi nghiên cứu dữ liệu và chức năng	6
1.4.2	Giải pháp công nghệ và Môi trường thực thi (Tech Stack)	6
1.5	Ý nghĩa thực tiễn đối với Ngành Du lịch số	7
2	Phân tích hệ thống	8
2.1	Phân tích Tác nhân (Actors) và Biểu đồ Use Case	8
2.1.1	Tác nhân Khách vãng lai (Guest)	8
2.1.2	Tác nhân Thành viên hệ thống (Authenticated User)	8
2.1.3	Tác nhân Quản trị viên tối cao (System Administrator)	10
2.2	Mô tả Chức năng Hệ thống và Tích hợp Mã nguồn	10
2.2.1	Module Khám phá và Sàng lọc Địa điểm (Location Exploration)	10
2.2.2	Module Chấm điểm và Tái tính toán Chỉ số (Rating & Metrics Recalculation)	11
2.2.3	Module Quản trị và Tự bảo vệ Hệ thống (Admin Self-Protection Logic)	12
2.3	Thiết kế Cơ sở Dữ liệu và Ràng buộc Toàn vẹn	13
2.3.1	Mô tả chi tiết các bảng dữ liệu cốt lõi	13
2.3.2	Chiến lược toàn vẹn dữ liệu: Chuyển dịch từ Soft Delete sang Cascade Cleanup	13
2.4	Phân tích Luồng Xử lý Dữ liệu động (Sequence & Workflow Logic)	14
2.4.1	Quy trình Đóng góp và Kiểm duyệt Nội dung (Review Workflow)	14
2.4.2	Quy trình Đồng bộ Hóa Danh sách Yêu thích (Favorite Toggle Logic)	14
3	Thiết kế và xây dựng	15
3.1	Kiến trúc Laravel MVC áp dụng trong hệ thống	15
3.1.1	Tầng Dữ liệu (Model & Eloquent ORM)	15
3.1.2	Tầng Điều khiển (Controller Logic)	15
3.1.3	Tầng Hiển thị (Blade Template Engine)	16

3.1.4	Tầng Chốt chặn Bảo mật (Middleware layered Architecture)	16
3.2	Các module chức năng và Hiện thực hóa Mã nguồn	16
3.2.1	Phân hệ Công cộng (Public Module)	16
3.2.2	Phân hệ Tương tác Thành viên (User Interaction Module)	17
3.2.3	Phân hệ Quản trị tối cao (Admin Control Module)	18
3.3	Thiết kế Giao diện Hệ thống	19
3.3.1	Kiến trúc Blade Layouting và Kế thừa Hệ thống Giao diện	19
3.3.2	Thiết kế Đáp ứng (Responsive Design) bằng Grid System	19
3.3.3	Hệ thống Thông báo Trạng thái và Phản hồi UI/UX trực quan . . .	20
4	Kết quả thực hiện và Đánh giá	21
4.1	Hình ảnh giao diện hệ thống	21
4.1.1	Giao diện phân hệ người dùng (Client-side)	21
4.1.2	Giao diện phân hệ quản trị viên (Admin-side)	22
4.2	Chức năng đã hoàn thành	23
4.3	Kiểm thử và Đánh giá hệ thống	24
4.3.1	Kiểm thử chức năng (Functional Testing)	24
4.3.2	Đánh giá khía cạnh an toàn thông tin	24
4.4	Hướng phát triển trong tương lai	24
4.2.	Phân chia công việc	24

Chương 1

Giới thiệu đề tài

1.1 Đặt vấn đề và Lý do lựa chọn đề tài

Trong kỷ nguyên Cách mạng Công nghiệp 4.0, sự hội tụ giữa công nghệ thông tin và truyền thông đã thúc đẩy sự hình thành của khái niệm "Du lịch số" (Digital Tourism). Ngành du lịch hiện đại không còn bó hẹp trong các phương thức tiếp cận truyền thống như tài liệu in ấn, bản đồ giấy hay các đại lý lữ hành trực tiếp. Thay vào đó, hành vi của du khách đã dịch chuyển hoàn toàn sang môi trường số. Trước khi quyết định đặt chân đến một vùng đất mới, người tiêu dùng có xu hướng tìm kiếm, sàng lọc và phân tích một khối lượng lớn thông tin trên mạng Internet để giảm thiểu rủi ro và tối ưu hóa trải nghiệm cá nhân.

Trong số các nguồn tài nguyên thông tin trực tuyến, nội dung do người dùng tự tạo (User-Generated Content - UGC) đóng một vai trò quyết định mang tính sống còn đối với danh tiếng và sự phát triển của các điểm đến. Các bài đánh giá (Reviews), bình luận (Comments) và điểm số (Ratings) từ những người đi trước mang lại giá trị thực tiễn, tính khách quan cao hơn nhiều so với các thông tin quảng cáo một chiều từ các đơn vị kinh doanh lữ hành. Sự tin cậy này xuất phát từ bản chất phi lợi nhuận và tính chân thực trong trải nghiệm của cộng đồng du lịch.

Tuy nhiên, việc xây dựng và vận hành một hệ thống quản lý dữ liệu đánh giá du lịch tại Việt Nam hiện nay đang đối mặt với nhiều thách thức lớn về mặt kỹ thuật và quản lý:

- Vấn nạn thông tin rác và đánh giá ảo: Hiện tượng các tổ chức hoặc cá nhân cố tình tạo tài khoản ảo để "spam" đánh giá tích cực nhằm trục lợi, hoặc hạ bệ đối thủ bằng những bình luận tiêu cực đang làm xói mòn niềm tin của người dùng.
- Hiệu năng và tính toàn vẹn dữ liệu: Các hệ thống cũ thường gặp khó khăn trong việc xử lý đồng bộ luồng dữ liệu lớn khi có hàng ngàn người dùng cùng tương tác, chấm điểm và tải lên các tệp tin đa phương tiện dung lượng cao.
- Cơ chế kiểm duyệt lỏng lẻo: Thiếu hụt một quy trình xử lý và phê duyệt nội dung khoa học trước khi hiển thị công khai ra môi trường bên ngoài.

Xuất phát từ thực trạng trên, đề tài "**Hệ thống quản lý và đánh giá địa điểm du lịch (Travel Review)**" được lựa chọn nhằm nghiên cứu, thiết kế và phát triển một giải pháp phần mềm toàn diện, giải quyết triệt để các bài toán tương tác thông tin trong ngành du lịch số. Ứng dụng được triển khai trên nền tảng framework Laravel 12 tiên tiến, kết hợp với ngôn ngữ PHP 8.2+, tận dụng các cơ chế tối ưu hóa cơ sở dữ liệu hiện đại để

tạo ra một không gian chia sẻ thông tin minh bạch, bảo mật và có giá trị thực tiễn cao cho cộng đồng.

1.2 Mục tiêu phát triển hệ thống

1.2.1 Mục tiêu tổng quát

Mục tiêu cốt lõi của đề tài là xây dựng hoàn chỉnh một ứng dụng Web thể hệ mới theo mô hình MVC (Model-View-Controller) cho phép quản lý, lưu trữ, điều hướng và kiểm duyệt toàn bộ luồng thông tin liên quan đến các địa điểm du lịch, bài đánh giá và tương tác của người dùng. Hệ thống phải đảm bảo tính mở về mặt kiến trúc để dễ dàng nâng cấp, có tốc độ phản hồi nhanh, giao diện thân thiện và có tính thực dụng cao đối với môi trường giáo dục và doanh nghiệp du lịch.

1.2.2 Mục tiêu cụ thể (Bám sát kiến trúc mã nguồn)

Để đạt được mục tiêu tổng quát, quá trình phân tích và lập trình mã nguồn của dự án đã hiện thực hóa cụ thể các mục tiêu kỹ thuật sau:

1. Xây dựng cơ chế xác thực và phân quyền đa tầng (Multi-tier Authentication): Triển khai hệ thống quản lý tài khoản thông qua các Middleware chuyên biệt của Laravel. Phân định rõ ràng ranh giới truy cập giữa khách vãng lai, thành viên hệ thống và ban quản trị tối cao, ngăn chặn tuyệt đối các lỗ hổng thực thi quyền lực chéo (Privilege Escalation).
2. Thiết kế và chuẩn hóa cơ sở dữ liệu quan hệ hướng thực thể: Xây dựng cấu trúc bảng vững chắc bằng công cụ Migration của Laravel. Thiết lập các chỉ mục (**index**) tại các cột có tần suất truy vấn cao như **role**, **status**, **region**, **category** để tối ưu hóa hiệu năng tìm kiếm dữ liệu lớn.
3. Hiện thực hóa quy trình kiểm duyệt nội dung hai bước an toàn: Lập trình logic nghiệp vụ sao cho mọi bài viết (**Review**) và bình luận (**Comment**) khi khởi tạo từ phía người dùng đều mặc định mang trạng thái ẩn '**pending**'. Nội dung chỉ được phép xuất bản (**Publish**) ra môi trường công cộng sau khi có thao tác kích hoạt **approve** một cách tường minh từ tài khoản có quyền **admin**.
4. Triển khai thuật toán tính toán điểm trung bình thời gian thực (Real-time Metric Calculation): Xây dựng cơ chế tự động hóa trong **RatingController**: ngay khi một thành viên gửi điểm số đánh giá mới (từ 1 đến 5 sao), hệ thống sẽ lập tức kích hoạt hàm tính tổng đại số và trung bình cộng (**avg()**), sau đó cập nhật trực tiếp kết quả làm tròn (**round**) vào cột **avg_rating** thuộc bảng **locations**, loại bỏ hoàn toàn độ trễ hiển thị.
5. Tối ưu hóa tài nguyên vật lý bằng giải pháp Cascade Cleanup: Thông qua việc tinh chỉnh hệ thống bằng bản vá loại bỏ cơ chế xóa mềm (**dropSoftDeletes**), dự án chuyển dịch sang sử dụng triệt để ràng buộc khóa ngoại **cascadeOnDelete()**. Giải pháp này giúp dọn dẹp sạch sẽ toàn bộ các bản ghi phụ thuộc khi một thực thể cha bị xóa, ngăn chặn tình trạng tràn ứ và mồ côi dữ liệu (Orphaned Rows) trong ổ đĩa cứng của máy chủ.

1.3 Đối tượng sử dụng và Phân quyền hệ thống

Kiến trúc định tuyến của hệ thống trong file `web.php` và `auth.php` đã phân rã toàn bộ người dùng tương tác thành 3 Actor (Tác nhân) riêng biệt với các giới hạn nghiệp vụ được kiểm soát nghiêm ngặt:

1.3.1 Khách vắng lai (Guest / Anonymous User)

Đây là những đối tượng tiếp cận hệ thống một cách tự do, chưa thực hiện định danh thông qua biểu mẫu đăng nhập. Nhóm này được hệ thống cấp quyền đọc một chiều (Read-only) đối với các thông tin mang tính đại chúng:

- Truy cập vào trang chủ, xem danh sách toàn bộ các địa điểm du lịch có trên hệ thống thông qua hàm `index()` của `LocationController`.
- Sử dụng bộ lọc tìm kiếm động để tra cứu địa điểm dựa trên từ khóa tên (`name`) hoặc vùng miền (`region`).
- Xem chi tiết từng địa điểm (`show()`), đọc các bài bình luận và bài viết đánh giá chuyên sâu từ các thành viên khác, với điều kiện các nội dung đó đã được admin phê duyệt (`status = 'approved'`).
- Khách vắng lai bị chặn hoàn toàn đối với các hành động tương tác thay đổi trạng thái database và được điều hướng đến các route `login` hoặc `register` thông qua Middleware `'guest'`.

1.3.2 Thành viên hệ thống (Authenticated User)

Sau khi trải qua quy trình xác thực danh tính hợp lệ, tài khoản người dùng được gán mã định danh duy nhất (`user_id`) và được bảo vệ bởi lớp rào chắn Middleware `'auth'`. Thành viên chính thức sở hữu toàn quyền tương tác và đóng góp nội dung cho hệ thống:

- Module Đánh giá & Bình luận: Khởi tạo bài viết review chi tiết thông qua `ReviewController`, viết bình luận phản hồi dưới các bài viết của người dùng khác.
- Module Chấm điểm & Yêu thích: Thực hiện chấm điểm trải nghiệm theo thang đo số nguyên (`score`). Hệ thống ràng buộc tính duy nhất thông qua cơ chế khóa phức hợp `unique(['user_id', 'location_id'])`, đảm bảo một người dùng không thể gian lận điểm số tại một địa điểm. Họ cũng có thể bật/tắt trạng thái lưu trữ địa điểm vào danh sách cá nhân qua cơ chế `toggle()` của `FavoriteController`.
- Module Cá nhân hóa: Cập nhật thông tin định danh cá nhân (Họ tên, Email) và kích hoạt lệnh xóa tài khoản vĩnh viễn khỏi hệ thống sau khi xác minh lại mật khẩu hiện tại (`current_password`).

1.3.3 Quản trị viên tối cao (System Administrator)

Là nhóm người dùng đặc quyền, sở hữu năng lực can thiệp toàn diện vào vòng đời dữ liệu của hệ thống. Quyền truy cập của Admin được bảo vệ kép bởi sự kết hợp của hai Middleware [`'auth'`, `'admin'`] trên toàn bộ nhóm định tuyến mang tiền tố `Route::prefix('admin')`. Nhiệm vụ của Admin bao gồm:

- Quản trị Thực thể Địa điểm: Sử dụng mẫu thiết kế Resource Controller (`AdminLocationController`) để thực hiện trọn vẹn quy trình CRUD (Thêm mới, Hiển thị, Sửa đổi, Xóa bỏ) các tọa độ, danh mục du lịch.
- Kiểm duyệt Nội dung Cộng đồng: Admin trực tiếp vận hành các hàm `approve()` và `reject()` đối với cả hai thực thể `Review` và `Comment`. Đây là chốt chặn tối cao giúp thanh lọc các nội dung không phù hợp trước khi đưa ra màn hình hiển thị công cộng.
- Quản lý và Điều phối Tài nguyên Nhân sự: Khởi tạo tài khoản mới cho nhân viên, quản lý danh sách người dùng, thay đổi mật khẩu cưỡng bức và thực hiện thao tác đóng/mở khóa trạng thái hoạt động của tài khoản (`toggleStatus()`) thông qua cột logic `is_active`. Hệ thống mã nguồn tích hợp sẵn cơ chế tự vệ (Self-Protection Logic), từ chối thực thi nếu một Admin cố tình tự khóa, tự xóa hoặc tự hạ quyền chính mình.

1.4 Phạm vi nghiên cứu và Giải pháp công nghệ

1.4.1 Phạm vi nghiên cứu dữ liệu và chức năng

Dự án tập trung giới hạn nghiên cứu vào luồng tương tác khép kín của mô hình mạng xã hội thu nhỏ chuyên ngành du lịch. Hệ thống không mở rộng sang các mảng thương mại điện tử phức tạp như thanh toán trực tuyến, đặt vé máy bay hay quản lý kho phòng khách sạn. Trọng tâm cốt lõi đặt vào việc tối ưu hóa cấu trúc cơ sở dữ liệu quan hệ giữa 5 thực thể chính: Người dùng, Địa điểm, Bài viết, Bình luận và Điểm số, đảm bảo tính đồng bộ dữ liệu tuyệt đối dưới các điều kiện ràng buộc khóa ngoại.

1.4.2 Giải pháp công nghệ và Môi trường thực thi (Tech Stack)

Mã nguồn của hệ thống được cấu hình, đóng gói và quản lý thông qua các công nghệ chuẩn công nghiệp, được thể hiện rõ nét trong tệp cấu hình hệ thống `composer.json`:

- Framework Laravel 12.0: Phiên bản kiến trúc mới nhất của hệ sinh thái Laravel, cung cấp nền tảng xử lý Request, Dependency Injection, Service Container mạnh mẽ và an toàn.
- Ngôn ngữ lập trình PHP ≥ 8.2 : Tận dụng các tính năng hiện đại mang tính đột phá của PHP như Định kiểu nghiêm ngặt (Strict Typing), Constructor Property Promotion, readonly properties, giúp mã nguồn trở nên sáng sủa, tối ưu hóa tốc độ biên dịch và giảm thiểu lỗi runtime.
- Công cụ quản lý thư viện Composer: Điều phối và liên kết chặt chẽ các gói thư viện phụ thuộc (Dependencies) như `laravel/framework`, `laravel/tinker` phục vụ môi trường thực thi lệnh CLI nhanh chóng.

- Hệ thống Thư viện Phát triển (Development Tools): Tích hợp `laravel/breeze` để dựng nhanh cấu trúc Scaffolding xác thực an toàn, bộ công cụ kiểm thử tự động `phpunit/phpunit` để viết các kịch bản Unit Test và Feature Test, đảm bảo độ ổn định của mã nguồn trước khi tích hợp liên tục (CI/CD).
- Tầng hiển thị (Frontend Layout): Engine Blade của Laravel chịu trách nhiệm kết xuất mã HTML kết hợp đồng bộ cấu trúc CSS linh hoạt, tạo nên một hệ thống giao diện mượt mà, phản hồi trực quan đối với các thông báo thành công (*success*) hay thông báo lỗi hệ thống từ phía Controller trả về.

1.5 Ý nghĩa thực tiễn đối với Ngành Du lịch số

Việc nghiên cứu và hiện thực hóa thành công đề tài hệ thống "Travel Review" mang lại những đóng góp thiết thực cho cả khía cạnh khoa học ứng dụng lẫn thực tế vận hành:

1. Đối với Cộng đồng Du khách và Người tiêu dùng: Hệ thống cung cấp một điểm đến thông tin đáng tin cậy, giúp du khách dễ dàng định vị các tọa độ du lịch uy tín thông qua hệ thống điểm số trung bình trực quan. Cơ chế bộ lọc vùng miền và công cụ tìm kiếm tối ưu giúp tiết kiệm tối đa thời gian lập kế hoạch lịch trình chuyến đi.
2. Đối với các Cơ quan và Đơn vị Quản lý Địa điểm: Hệ thống đóng vai trò như một kênh tiếp nhận phản hồi (Feedback Loop) đa chiều. Thông qua các chỉ số thống kê biến động số lượng bài đánh giá theo dòng thời gian trên trang Dashboard của Admin, các nhà quản lý có thể thấu hiểu sâu sắc mức độ hài lòng, thị hiếu và xu hướng dịch chuyển của du khách. Từ đó, đưa ra các quyết định cải thiện chất lượng dịch vụ hoặc đầu tư hạ tầng một cách khoa học, dựa trên dữ liệu thực tế (Data-Driven Decision Making).
3. Đối với Đội ngũ Phát triển Phần mềm (Khía cạnh Học thuật): Dự án là một minh chứng thực tế cho việc ứng dụng các nguyên lý thiết kế phần mềm bền vững (SOLID Principles) trong môi trường framework Laravel hiện đại. Việc giải quyết triệt để các bài toán thực tế như chống spam đánh giá, xử lý dọn dẹp bộ nhớ cơ sở dữ liệu bằng Cascade Cleanup và bảo vệ hệ thống bằng Middleware đa lớp là những kinh nghiệm thực chiến vô giá trong việc thiết kế các hệ thống phần mềm quy mô lớn sau này.

Chương 2

Phân tích hệ thống

2.1 Phân tích Tác nhân (Actors) và Biểu đồ Use Case

Để thiết kế một kiến trúc phần mềm an toàn và hướng chức năng, việc định hình các tác nhân (Actors) và ranh giới hệ thống là bước tiên quyết. Hệ thống "Travel Review" phân rã không gian tương tác thành 3 tác nhân cốt lõi với các biên đặc quyền tuyến tính, được ràng buộc chặt chẽ thông qua cơ chế kiểm soát truy cập phân tầng.

2.1.1 Tác nhân Khách vãng lai (Guest)

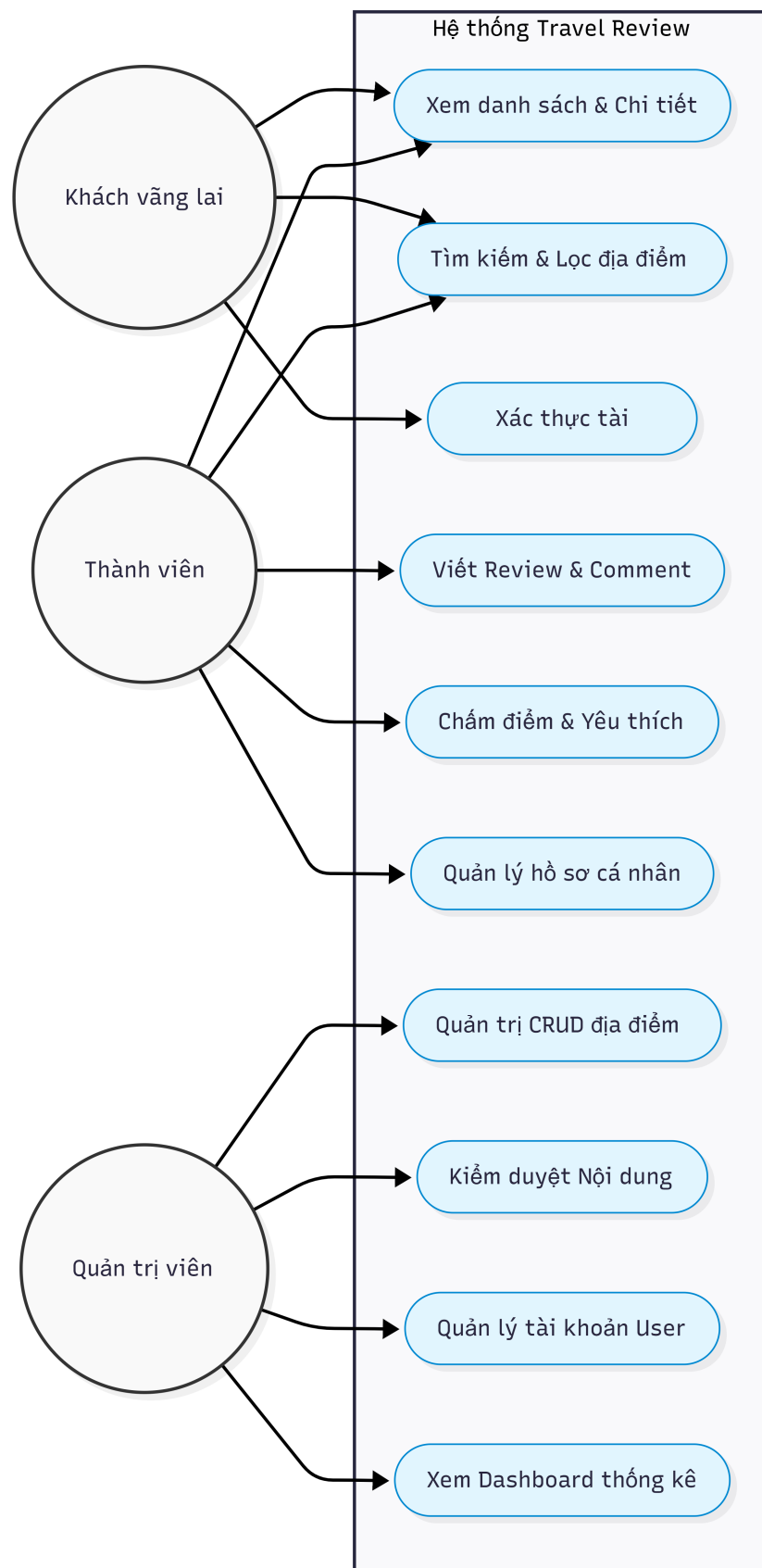
Khách vãng lai đại diện cho những thực thể chưa thực hiện định danh (Unauthenticated Entities). Biên tương tác của tác nhân này được giới hạn hoàn toàn ở các truy vấn đọc dữ liệu không thay đổi trạng thái (Idempotent Read Operations). Mục tiêu của Guest là tìm kiếm và tiếp cận thông tin du lịch đại chúng. Hệ thống cho phép Guest thực hiện các Use case:

- Use case Duyệt danh sách địa điểm: Tiếp cận toàn bộ kho dữ liệu địa điểm đã được khởi tạo trên hệ thống.
- Use case Tìm kiếm và Lọc động: Khai thác bộ lọc dữ liệu theo từ khóa tên địa điểm hoặc phân vùng địa lý để thu hẹp phạm vi tìm kiếm.
- Use case Xem chi tiết địa điểm: Đọc thông tin chi tiết cấu trúc điểm đến, bao gồm cả việc gián tiếp tiếp cận các bài đánh giá (Reviews) và bình luận (Comments) dưới điều kiện các thực thể phụ thuộc này đã chuyển trạng thái phê duyệt.

2.1.2 Tác nhân Thành viên hệ thống (Authenticated User)

Thành viên là những thực thể đã vượt qua chốt chặn xác thực danh tính cá nhân và được hệ thống cấp phát một phiên làm việc (Session) hợp lệ. Tác nhân này vận hành các nghiệp vụ sản sinh nội dung (User-Generated Content - UGC) và cá nhân hóa trải nghiệm:

- Use case Đăng bài đánh giá chuyên sâu (Submit Review): Soạn thảo văn bản đánh giá kèm tiêu đề.
- Use case Chấm điểm trải nghiệm (Submit Rating): Thực hiện bỏ phiếu điểm số từ 1 đến 5 sao, kích hoạt trực tiếp chuỗi tính toán toán học phía backend.



Hình 2.1: Biểu đồ Use Case tổng quát của hệ thống Travel Review

- Use case Bình luận tương tác (Submit Comment): Tham gia thảo luận bên dưới các bài đánh giá của thành viên khác.
- Use case Quản lý danh sách yêu thích (Toggle Favorite): Bật/tắt trạng thái lưu trữ địa điểm vào kho lưu trữ cá nhân.

2.1.3 Tác nhân Quản trị viên tối cao (System Administrator)

Quản trị viên là tác nhân sở hữu đặc quyền tối thượng, chịu trách nhiệm điều phối vòng đời của toàn bộ các thực thể dữ liệu và giám sát an ninh hệ thống. Phạm vi Use case bao gồm:

- Use case Quản trị CRUD Địa điểm: Khởi tạo, cập nhật thông tin tọa độ và cấu trúc danh mục các điểm đến du lịch.
- Use case Kiểm duyệt Nội dung (Approve/Reject): Đóng vai trò chốt chặn phê duyệt tối cao đối với hai thực thể **Review** và **Comment** trước khi xuất bản rộng rãi.
- Use case Quản lý Tài khoản (User Management): Điều khiển trạng thái hoạt động (**is_active**), thay đổi mật khẩu cưỡng bức và xóa bỏ tài khoản vi phạm chính sách.

2.2 Mô tả Chức năng Hệ thống và Tích hợp Mã nguồn

Hệ thống hiện thực hóa các chức năng nghiệp vụ thông qua việc phân rã logic xử lý thành các Controller chuyên biệt, bám sát kiến trúc hướng đối tượng của Laravel. Dưới đây là phân tích chi tiết các module chức năng trọng tâm kèm mã nguồn backend thực tế:

2.2.1 Module Khám phá và Sàng lọc Địa điểm (Location Exploration)

Nghiệp vụ này giải quyết bài toán truy vấn dữ liệu động, tối ưu hóa hiệu năng bằng cơ chế Eager Loading để đếm số lượng bài đánh giá đã được phê duyệt mà không làm phát sinh lỗi truy vấn N+1. Đồng thời, mã nguồn tích hợp bộ lọc điều kiện (Conditional Clauses) để xử lý các tham số tìm kiếm **q** và **region** từ URL Request.

Mã nguồn thực thi logic này nằm trong hàm **index** của **LocationController**:

```
public function index(Request $request)
{
    $search = $request->string('q')->toString();
    $region = $request->string('region')->toString();

    $query = Location::query()
        ->withCount(['reviews' => function ($reviewQuery) {
            $reviewQuery->where('status', 'approved');
        }]);

    if ($search !== '') {
```

```

    $query->where(function ($filterQuery) use ($search) {
        $filterQuery->where('name', 'like', "%{$search}%")
        ->orWhere('region', 'like', "%{$search}%");
    });
}

if ($region !== '') {
    $query->where('region', $region);
}

$locations = $query->orderByDesc('created_at')
    ->paginate(9)
    ->withQueryString();

return view('locations.index', compact('locations', 'search', 'region'));
}

```

2.2.2 Module Chấm điểm và Tái tính toán Chỉ số (Rating & Metrics Recalculation)

Đây là một trong những nghiệp vụ phức tạp và quan trọng nhất của hệ thống. Luồng xử lý yêu cầu tính toán vẹn dữ liệu rất cao:

1. Hệ thống thực hiện kiểm tra thực thể xem User hiện tại đã chấm điểm cho địa điểm này chưa. Nếu đã tồn tại, lập tức ném lỗi ngăn chặn hành vi spam dữ liệu.
2. Nếu hợp lệ, một bản ghi mới được khởi tạo trong bảng `ratings`.
3. Ngay sau đó, hệ thống gọi hàm gom cụm đại số `avg('score')` để tính điểm trung bình mới của toàn bộ địa điểm. Điểm số này được ép kiểu và làm tròn về 2 chữ số thập phân bằng hàm `round()` trước khi cập nhật trực tiếp vào trường `avg_rating` của bảng `locations`.

Mã nguồn thực tế kiểm soát giao dịch này trong `RatingController`:

```

public function store(StoreRatingRequest $request)
{
    $data = $request->validated();

    $existing = Rating::where('location_id', $data['location_id'])
        ->where('user_id', $request->user()->id)
        ->first();

    if ($existing) {
        return back()->withErrors(['score' => 'You already rated this location.']);
    }

    Rating::create([
        'score' => $data['score'],
        'comment' => $data['comment'] ?? null,
    ]);
}

```

```

        'user_id' => $request->user()->id,
        'location_id' => $data['location_id'],
    ]);

    $avg = Rating::where('location_id', $data['location_id'])->avg('score');
    Location::where('id', $data['location_id'])->update([
        'avg_rating' => round((float) $avg, 2),
    ]);

    return back()->with('success', 'Rating submitted.');
```

2.2.3 Module Quản trị và Tự bảo vệ Hệ thống (Admin Self-Protection Logic)

Để ngăn chặn các lỗi hỏng vận hành do sai sót của con người, phân hệ quản trị người dùng trong `Admin/UserController` được thiết kế với tư duy an ninh nghiêm ngặt. Khi một Admin thực hiện chỉnh sửa, khóa trạng thái hoặc xóa tài khoản, hệ thống sẽ chạy các mệnh đề kiểm tra để đối chiếu ID của đối tượng mục tiêu với ID của Admin đang đăng nhập (`auth()->id()`). Hệ thống sẽ lập tức từ chối thực thi và đẩy thông báo lỗi nếu Admin cố tình tự tước quyền admin hoặc tự khóa chính mình.

Mình chứng thực tế được trích xuất từ hàm `update` trong `Admin/UserController`:

```

public function update(Request $request, User $user)
{
    $data = $request->validate([
        'name' => ['required', 'string', 'max:255'],
        'email' => ['required', 'string', 'email', 'max:255',
            Rule::unique('users', 'email')->ignore($user->id)],
        'role' => ['required', Rule::in(['admin', 'user'])],
        'is_active' => ['nullable', 'boolean'],
    ]);

    if ($user->id === auth()->id() && ! ($data['is_active'] ?? false)) {
        return back()->withErrors([
            'status' => 'You cannot deactivate your own account.'
        ])->withInput();
    }

    if ($user->id === auth()->id() && ($data['role'] ?? 'user') !== 'admin') {
        return back()->withErrors([
            'status' => 'You cannot remove your own admin role.'
        ])->withInput();
    }

    $user->update([
        'name' => $data['name'],
        'email' => $data['email'],
        'role' => $data['role'],
    ]);
}
```

```

        'is_active' => (bool) ($data['is_active'] ?? false),
    ]);

    return redirect()->route('admin.users.index')->with('success', 'User updated.')
}

```

2.3 Thiết kế Cơ sở Dữ liệu và Ràng buộc Toàn vẹn

Kiến trúc dữ liệu của hệ thống được xây dựng dựa trên mô hình quan hệ chuẩn hóa cao để triệt tiêu tình trạng dư thừa dữ liệu và đảm bảo tính nhất quán.

2.3.1 Mô tả chi tiết các bảng dữ liệu cốt lõi

- Bảng **users**: Trọng tâm quản lý định danh. Chứa các trường cơ bản như **name**, **email** (được gán thuộc tính **unique** chống trùng lặp), chuỗi băm **password**, phân quyền **role** với giá trị mặc định là **'user'** và cờ logic **is_active** kiểm soát trạng thái đăng nhập. Cột **role** được thiết lập thuộc tính **index()** để đẩy nhanh tốc độ lọc phân quyền của middleware.
- Bảng **locations**: Lưu trữ thực thể địa điểm du lịch. Trường **slug** mang tính duy nhất giúp tối ưu SEO đường dẫn. Trường **avg_rating** có kiểu dữ liệu là **decimal(3,2)** cho phép lưu trữ điểm số chính xác đến hai chữ số phần thập phân (ví dụ: 4.67).
- Bảng **reviews** và **comments**: Lưu giữ nội dung tương tác. Cả hai bảng đều sở hữu trường **status** mang giá trị mặc định là **'pending'** kèm chỉ mục **index()** để phục vụ riêng cho các truy vấn kiểm duyệt của Admin.

2.3.2 Chiến lược toàn vẹn dữ liệu: Chuyển dịch từ Soft Delete sang Cascade Cleanup

Một điểm đặc biệt trong thiết kế hệ thống là cấu trúc xử lý vòng đời thực thể phụ thuộc. Ban đầu, các bảng được thiết lập thuộc tính xóa mềm **softDeletes()**. Tuy nhiên, để tối ưu dung lượng lưu trữ vật lý và đảm bảo dọn dẹp sạch sẽ dữ liệu rác, một bản vá hệ thống (**Migration cleanup**) đã được triển khai để loại bỏ triệt để xóa mềm:

```

public function up(): void
{
    Schema::table('comments', function (Blueprint $table) {
        if (Schema::hasColumn('comments', 'deleted_at')) {
            $table->dropSoftDeletes();
        }
    });
    // Thao tác tương tự được thực thi cho reviews, locations, users
}

```

Thay vào đó, hệ thống chuyển giao toàn bộ trách nhiệm đồng bộ cho ràng buộc khóa ngoại cứng ở tầng cơ sở dữ liệu thông qua khai báo **constrained()->cascadeOnDelete()**.

Khi một tài khoản người dùng hoặc một địa điểm bị xóa bỏ hoàn toàn khỏi hệ thống, toàn bộ các bài đánh giá, điểm số, bình luận và trạng thái yêu thích liên quan sẽ tự động bị xóa sạch theo cơ chế thác đổ (Cascading Delete), duy trì trạng thái sạch tuyệt đối cho hệ quản trị cơ sở dữ liệu quan hệ.

2.4 Phân tích Luồng Xử lý Dữ liệu động (Sequence & Workflow Logic)

Dựa trên cấu trúc mã nguồn đã phân tích, luồng xử lý thông tin được chia thành các kịch bản động tuyến tính khép kín:

2.4.1 Quy trình Đóng góp và Kiểm duyệt Nội dung (Review Workflow)

1. Giai đoạn khởi tạo: Thành viên truy cập biểu mẫu và gửi dữ liệu lên route `reviews.store`. Hệ thống tiếp nhận, kiểm tra tệp tin đa phương tiện và lưu trữ hình ảnh vào đĩa vật lý của máy chủ (`store('reviews', 'public')`). Bài đánh giá được ghi nhận với trạng thái `'pending'`.
2. Giai đoạn đóng băng dữ liệu: Tại thời điểm này, bài viết nằm trong trạng thái ẩn. Hàm `show()` tại `LocationController` khi kết xuất giao diện công cộng sẽ sử dụng mệnh đề `where('status', 'approved')` để chủ động loại bỏ bản ghi này ra khỏi tầm nhìn của Guest và các User khác.
3. Giai đoạn phê duyệt: Admin truy cập phân hệ quản trị, gọi hàm `approve()` được khai báo tại hệ thống route quản trị để cập nhật trạng thái bản ghi thành `'approved'`. Ngay lập tức, bài viết được đưa vào luồng kết xuất công cộng công khai.

2.4.2 Quy trình Đồng bộ Hóa Danh sách Yêu thích (Favorite Toggle Logic)

Hệ thống không sử dụng hai hàm riêng biệt cho việc thêm và xóa địa điểm yêu thích, mà tối ưu thành một luồng xử lý tuần hoàn duy nhất thông qua hàm `toggle()` của `FavoriteController`. Khi người dùng nhấn nút tương tác trên giao diện, hệ thống sẽ thực hiện một câu lệnh truy vấn điều kiện để kiểm tra xem bản ghi liên kết giữa `user_id` và `location_id` đã tồn tại trong bảng `favorites` chưa. Nếu đã tồn tại, hệ thống chạy lệnh `delete()` để gỡ bỏ. Nếu chưa tồn tại, hệ thống chạy lệnh `create()` để thiết lập liên kết. Toàn bộ quy trình kết thúc bằng lệnh `back()` để làm mới giao diện tại chỗ kèm thông báo trạng thái trực quan.

Chương 3

Thiết kế và xây dựng

3.1 Kiến trúc Laravel MVC áp dụng trong hệ thống

Hệ thống "Travel Review" được phát triển dựa trên mô hình kiến trúc Model - View - Controller (MVC) chuẩn mực của framework Laravel 12. Việc áp dụng kiến trúc này giúp phân tách rõ ràng giữa các tầng dữ liệu, logic nghiệp vụ và giao diện hiển thị, từ đó tối ưu hóa quy trình bảo trì, mở rộng hệ thống và tăng cường hiệu năng xử lý request.

3.1.1 Tầng Dữ liệu (Model & Eloquent ORM)

Trong hệ thống, tầng Model không chỉ đại diện cho các bảng trong cơ sở dữ liệu quan hệ mà còn là nơi thiết lập các mối quan hệ (Relationships) logic và các ràng buộc toàn vẹn dữ liệu ở tầng ứng dụng. Các Model kế thừa từ lớp Eloquent Model kế thừa gián tiếp thông qua lớp cơ sở của framework, cho phép lập trình viên tương tác với cơ sở dữ liệu bằng cú pháp hướng đối tượng thay vì viết các câu lệnh SQL thuần túy.

Hệ thống quản lý và liên kết chặt chẽ các thực thể dữ liệu thông qua các mối quan hệ Eloquent:

- Mối quan hệ Một - Nhiều (One-to-Many): Một địa điểm (**Location**) có thể sở hữu nhiều bài đánh giá (**Review**) và nhiều lượt chấm điểm (**Rating**). Tương tự, một bài viết đánh giá (**Review**) sẽ chứa nhiều bình luận (**Comment**) phụ thuộc. Các mối quan hệ này được cấu hình qua hàm `hasMany()` trong Model cha và hàm `belongsTo()` trong Model con.
- Mối quan hệ Nhiều - Nhiều (Many-to-Many): Được hiện thực hóa thông qua các bảng trung gian như **favorites** và **ratings**. Một người dùng (**User**) có thể yêu thích nhiều địa điểm (**Location**) và một địa điểm cũng có thể được lưu bởi nhiều người dùng khác nhau.

3.1.2 Tầng Điều khiển (Controller Logic)

Tầng Controller chịu trách nhiệm tiếp nhận các HTTP Request được chuyển hướng từ hệ thống định tuyến (Routing), thực hiện xử lý logic nghiệp vụ, tương tác với tầng Model để lấy dữ liệu và cuối cùng quyết định kết xuất ra giao diện View phù hợp hoặc trả về một phản hồi chuyển hướng (Redirect Response).

Trong dự án này, hệ thống Controller được quy hoạch rõ ràng thành hai phân hệ lớn:

- User-facing Controllers: Đặt trực tiếp trong thư mục `app/Http/Controllers/`, xử lý luồng tương tác của khách vãng lai và thành viên như đọc thông tin, tìm kiếm, lưu yêu thích, chấm điểm và viết review.
- Admin-specific Controllers: Đặt trong không gian tên biệt lập `App\Http\Controllers\Admin`, quản trị toàn diện hệ thống thông qua các Resource Controller mã nguồn mở giúp chuẩn hóa luồng gọi hàm CRUD.

3.1.3 Tầng Hiển thị (Blade Template Engine)

Tầng View sử dụng công cụ Blade Template Engine tích hợp sẵn của Laravel. Blade cho phép lập trình viên viết mã HTML kết hợp cấu trúc lặp, điều kiện động của PHP một cách tối giản thông qua các cú pháp đặc trưng như `@extends`, `@section`, `@forelse`, và `@if`.

Kiến trúc View của dự án được xây dựng theo mô hình phân tầng layout: Một layout tổng thể `layouts.app` định nghĩa cấu trúc khung của trang web (Header, Navigation, Sidebar, Footer). Các view thành phần (như trang danh sách địa điểm, trang chi tiết) sẽ kế thừa layout này thông qua từ khóa `@extends('layouts.app')` và chỉ tập trung định nghĩa nội dung thay đổi bên trong khối `@section('content')`.

3.1.4 Tầng Chốt chặn Bảo mật (Middleware layered Architecture)

Middleware đóng vai trò như một bộ lọc trung gian kiểm soát luồng di chuyển của HTTP Request trước khi nó có quyền tiếp cận vào logic bên trong của Controller. Dự án triển khai kiến trúc bảo mật 3 tầng nghiêm ngặt nhằm bảo vệ tài nguyên hệ thống:

1. Middleware `'guest'`: Chặn các tài khoản đã đăng nhập tiếp cận lại các trang mang tính khởi tạo định danh như trang Đăng ký (`register`) hay Đăng nhập (`login`) nhằm bảo toàn trạng thái phiên hoạt động.
2. Middleware `'auth'`: Đảm bảo request đến từ một người dùng đã được định danh hợp lệ. Nếu chưa đăng nhập, hệ thống lập tức chặn đứng và điều hướng người dùng về trang login. Middleware này bảo vệ các tính năng sản sinh nội dung như `reviews.store`, `ratings.store`, và `comments.store`.
3. Middleware `'admin'`: Lớp phòng thủ tối cao, kết hợp cùng lớp xác thực [`'auth'`, `'admin'`] để bảo vệ toàn bộ phân hệ quản trị `Route::prefix('admin')`. Middleware này thực hiện kiểm tra giá trị cột `role` trong bảng `users`. Request chỉ được thông qua nếu giá trị chuỗi trả về khớp chính xác với hằng số `'admin'`, ngăn chặn triệt để mọi hành vi xâm nhập trái phép vào hệ thống quản lý dữ liệu.

3.2 Các module chức năng và Hiện thực hóa Mã nguồn

3.2.1 Phân hệ Công cộng (Public Module)

Phân hệ công cộng được thiết kế hướng tới tối ưu hóa trải nghiệm tìm kiếm và tra cứu thông tin của du khách. Điểm nhấn kỹ thuật tại `LocationController` là việc giải quyết

bài toán hiệu năng thông qua cơ chế **Eager Loading** và tính toán thống kê gián tiếp.

Logic Hiển thị và Sàng lọc Địa điểm động

Tại hàm `index()`, thay vì truy vấn toàn bộ dữ liệu thô gây quá tải bộ nhớ, hệ thống sử dụng phương thức `withCount()` để đếm số lượng bài review thuộc về địa điểm đó, với điều kiện bài viết đã được phê duyệt (`status = 'approved'`). Logic này giúp loại bỏ hoàn toàn lỗi hỏng hiệu năng N+1 query nguy hiểm trong thiết kế ứng dụng web lữ hành. Đồng thời, các câu lệnh kiểm tra chuỗi động (`$search !== ''`) cho phép lọc dữ liệu đa điều kiện theo tên (`name`) hoặc khu vực (`region`) trực tiếp từ thanh tìm kiếm.

Logic Kết xuất Chi tiết Thực thể Phức hợp

Khi người dùng truy cập vào trang chi tiết của một địa điểm du lịch thông qua cơ chế liên kết Implicit Route Model Binding, hàm `show()` sẽ tự động kích hoạt phương thức `load()` để nạp đồng bộ các mối quan hệ lồng nhau (Nested Relations):

```
$location->load([
    'user',
    'ratings',
    'reviews' => function ($reviewQuery) {
        $reviewQuery->where('status', 'approved')
        ->with(['user', 'comments' => function ($commentQuery) {
            $commentQuery->where('status', 'approved')->with('user');
        }]);
    },
]);
```

Đoạn mã trên đảm bảo rằng chỉ những bài Review đã duyệt mới được tải lên giao diện, và đi kèm với mỗi bài review là danh sách các bình luận (`Comments`) cũng đã ở trạng thái phê duyệt. Đồng thời, hàm cũng thực hiện kiểm tra trạng thái đăng nhập của người dùng hiện tại (`auth()->check()`) để xác định xem họ đã từng chấm điểm cho địa điểm này chưa (`$userRating`) và địa điểm này đã nằm trong danh mục yêu thích (`$isFavorite`) của họ hay chưa.

3.2.2 Phân hệ Tương tác Thành viên (User Interaction Module)

Thành viên sau khi đăng nhập được cung cấp các công cụ tương tác mạnh mẽ thông qua các Controller xử lý luồng dữ liệu POST, đảm bảo tính an toàn bằng các lớp Form Request Validation chuyên biệt.

Cơ chế Tính toán Chỉ số Rating thời gian thực

Trong `RatingController`, hệ thống xử lý một luồng nghiệp vụ khép kín từ kiểm tra ràng buộc đến tái cấu trúc điểm số:

```
$existing = Rating::where('location_id', $data['location_id'])
    ->where('user_id', $request->user()->id)
    ->first();
```

```
if ($existing) {
    return back()->withErrors(['score' => 'You already rated this location.']);
}
```

Đoạn mã trên là lớp phòng thủ nghiệp vụ, ngăn chặn người dùng cố ý gửi request giả lập để chấm điểm nhiều lần. Sau khi bản ghi chấm điểm mới được khởi tạo thành công, hệ thống lập tức thực hiện tính toán lại giá trị trung bình cộng thông qua hàm gom cụm đại số `avg('score')` của Eloquent. Kết quả sau đó được ép kiểu sang số thực (`float`) và làm tròn chính xác đến 2 chữ số phần thập phân bằng hàm `round()` của PHP trước khi lưu lại vào bảng địa điểm. Điều này giúp dữ liệu điểm số trung bình của các tọa độ du lịch luôn được cập nhật chính xác và đồng bộ theo thời gian thực.

Luồng Khởi tạo Bài viết Đánh giá đính kèm Tập tin

Đối với chức năng viết bài Review, `ReviewController@store` tiếp nhận luồng dữ liệu bao gồm cả tập tin hình ảnh đa phương tiện do người dùng tải lên. Hệ thống kiểm tra thông qua điều kiện `$request->hasFile('image')`, sau đó sử dụng phương thức `store('reviews', 'public')` để tự động bấm tên tập tin nhằm chống trùng lặp, đẩy tập tin vào ổ đĩa vật lý của server và chỉ lưu trữ chuỗi đường dẫn tương đối vào database. Bài viết sau đó được gán cứng trạng thái ẩn `'pending'` để chuyển giao sang phân hệ kiểm duyệt.

3.2.3 Phân hệ Quản trị tối cao (Admin Control Module)

Phân hệ quản trị được xây dựng tập trung vào năng lực kiểm soát dữ liệu toàn diện và cơ chế tự bảo mật hệ thống trước các hành vi thao tác sai lệch.

Xử lý Dữ liệu Chuỗi thời gian (Time-series Data Processing) trên Dashboard

Tại `DashboardController`, trang tổng quan không chỉ đếm tổng số lượng thực thể tính đơn thuần bằng hàm `count()` mà còn xử lý các tập dữ liệu động phục vụ cho việc vẽ biểu đồ thống kê. Sử dụng thư viện `Carbon\Carbon`, hệ thống xác định mốc thời gian lùi về 5 tháng trước từ thời điểm hiện tại để tạo ra một khoảng thời gian liên tục gồm 6 tháng gần nhất.

Mã nguồn thực hiện truy vấn nhóm theo thời gian (Group By Time Query):

```
$reviewStats = Review::selectRaw(
    "DATE_FORMAT(created_at, '%Y-%m') as month, COUNT(*) as total"
)
->where('created_at', '>=', $start)
->groupBy('month')
->orderBy('month')
->get()
->pluck('total', 'month');
```

Hàm `pluck()` chuyển đổi tập kết quả thành một mảng Key-Value với Key là chuỗi định dạng năm-tháng và Value là tổng số bài viết. Hệ thống sử dụng hàm `array_map` kết hợp mệnh đề null-coalescing (`?? 0`) để duyệt qua danh sách các tháng, đảm bảo rằng nếu tháng nào không có bài viết review nào, biểu đồ vẫn nhận giá trị số nguyên 0 thay vì bị lỗi trống dữ liệu hoặc sai lệch tọa độ trục hoành.

Hiện thực hóa Logic Tự bảo vệ Hệ thống của Admin

Trong quản lý nhân sự tại Admin/UserController, mã nguồn cấu trúc các mệnh đề điều kiện logic vô cùng nghiêm ngặt nhằm ngăn chặn các tình huống xung đột quản trị. Cụ thể, hệ thống chặn đứng hành vi "tự sát tài khoản" thông qua các chốt chặn kiểm tra định danh chéo:

```
if ($user->id === auth()->id() && ! ($data['is_active'] ?? false)) {
    return back()->withErrors([
        'status' => 'You cannot deactivate your own account.'
    ])->withInput();
}

if ($user->id === auth()->id() && ($data['role'] ?? 'user') !== 'admin') {
    return back()->withErrors([
        'status' => 'You cannot remove your own admin role.'
    ])->withInput();
}
```

Kiến trúc này đảm bảo một Admin đang đăng nhập không thể tự khóa trạng thái hoạt động của mình (`is_active = false`) và không thể tự hạ quyền quản trị của chính mình xuống thành thành viên thông thường (`role = 'user'`). Logic bảo vệ tương tự cũng được áp dụng tại các hàm `toggleStatus()` và `destroy()`, ném lỗi ngay lập tức nếu Admin cố tình khóa hoặc xóa chính mình khỏi cơ sở dữ liệu.

3.3 Thiết kế Giao diện Hệ thống

Giao diện của hệ thống "Travel Review" được thiết kế tối ưu hóa theo ngôn ngữ thiết kế tối giản, tập trung vào tính tương tác trực quan và bố cục phân vùng thông tin khoa học.

3.3.1 Kiến trúc Blade Layouting và Kế thừa Hệ thống Giao diện

Hệ thống sử dụng tệp layout tổng thể cấu trúc khung, tích hợp thư viện phông chữ trực tuyến `instrument-sans` từ hệ thống Bunny Fonts để mang lại trải nghiệm hiển thị chữ viết sắc nét, hiện đại. Các thành phần giao diện tĩnh hoặc các biểu mẫu nhỏ được module hóa thành các tệp tin thành phần (Partial Views) và được nhúng động vào trang thông qua từ khóa `@include()`. Ví dụ, trang chỉnh sửa hồ sơ `profile.edit` sử dụng lệnh nhúng để tích hợp độc lập form cập nhật thông tin cá nhân và form xóa tài khoản:

```
@include('profile.partials.update-profile-information-form')
@include('profile.partials.delete-user-form')
```

Kiến trúc phân rã này giúp mã nguồn giao diện cực kỳ sạch sẽ, dễ nâng cấp giao diện của từng form mà không làm ảnh hưởng đến cấu trúc dàn trang chung của toàn bộ hệ thống.

3.3.2 Thiết kế Đáp ứng (Responsive Design) bằng Grid System

Tầng View của ứng dụng sử dụng hệ thống lưới (Grid System) linh hoạt để tự động co giãn không gian hiển thị dựa trên kích thước màn hình thiết bị truy cập (Desktop, Máy tính bảng, Điện thoại di động).

Tại trang chi tiết địa điểm `locations.show`, không gian hiển thị được phân bổ theo tỷ lệ lưới thông minh:

- Cột hiển thị thông tin thực thể (`col-lg-5`): Chiếm 5 phần trên tổng số 12 phần chiều rộng màn hình lớn, dùng để gắn cố định hình ảnh đại diện lớn (`location-hero`), địa chỉ, vùng miền, phân loại và điểm đánh giá trung bình kèm nút tương tác lưu yêu thích.
- Cột nội dung tương tác cộng đồng (`col-lg-7`): Chiếm 7 phần còn lại, cung cấp không gian rộng rãi để hiển thị văn bản mô tả chi tiết địa điểm, tiếp theo là form chấm điểm nhanh, danh sách các bài viết Review dài và luồng bình luận thảo luận bên dưới.

Trên các màn hình thiết bị di động cỡ nhỏ, hệ thống lưới sẽ tự động phản hồi bằng cách xếp chồng hai cột này theo chiều dọc (`flex-col-reverse` hoặc phá vỡ hàng ngang), giúp du khách không phải thực hiện thao tác cuộn ngang màn hình gây bất tiện.

3.3.3 Hệ thống Thông báo Trạng thái và Phản hồi UI/UX trực quan

Để tăng cường tính thân thiện trong trải nghiệm người dùng (UX), hệ thống tích hợp các khối hiển thị thông báo trạng thái động (Flash Sessions). Khi một Controller xử lý thành công một tác vụ thay đổi dữ liệu, lệnh `with('success', '...')` sẽ đẩy một thông điệp ngắn vào vùng nhớ session tạm thời. Tại tầng View, Blade Engine kiểm tra sự tồn tại của session này để kết xuất ra các hộp thông báo lỗi (`alert-danger`) hoặc thông báo thành công (`alert-success`) mang màu sắc Bootstrap chuẩn mực.

Đối với các trường hợp lỗi dữ liệu đầu vào (Validation Errors), hệ thống tự động giữ lại giá trị cũ thông qua hàm `old()` tại các thẻ input, giúp người dùng không phải nhập lại toàn bộ thông tin từ đầu nếu xảy ra lỗi trong quá trình soạn thảo.

Chương 4

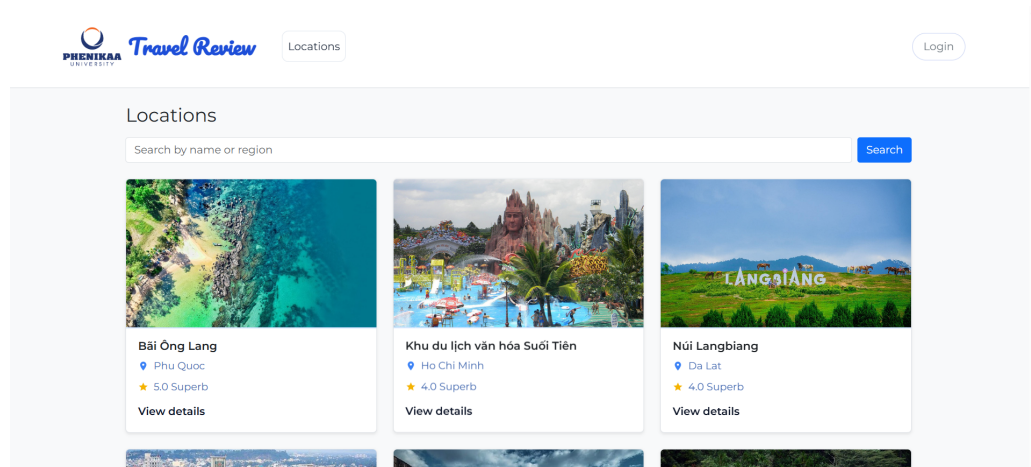
Kết quả thực hiện và Đánh giá

4.1 Hình ảnh giao diện hệ thống

Trong mục này, báo cáo trình bày các hình ảnh minh họa giao diện thực tế của hệ thống sau khi đã hoàn thiện lập trình và triển khai. Giao diện được thiết kế theo phong cách hiện đại, tối giản và tối ưu hóa trải nghiệm người dùng (UX/UI).

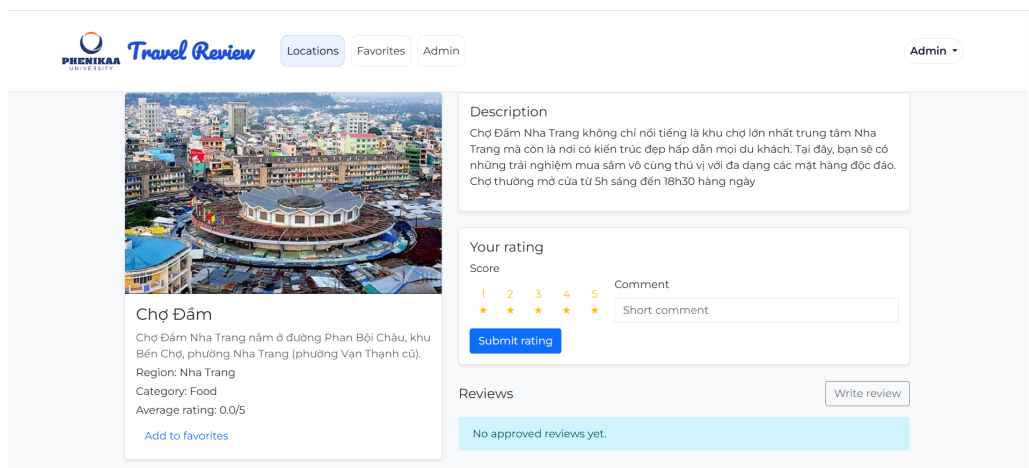
4.1.1 Giao diện phân hệ người dùng (Client-side)

Phân hệ dành cho người dùng cuối tập trung vào việc cung cấp thông tin trực quan, tìm kiếm nhanh chóng và tương tác mượt mà.



Hình 4.1: Giao diện Trang chủ hệ thống - Hiện thị danh mục điểm đến xu hướng

Mô tả: Trang chủ tích hợp thanh tìm kiếm thông minh, danh mục các địa điểm du lịch nổi bật và hiển thị các bài đánh giá mới nhất từ cộng đồng.

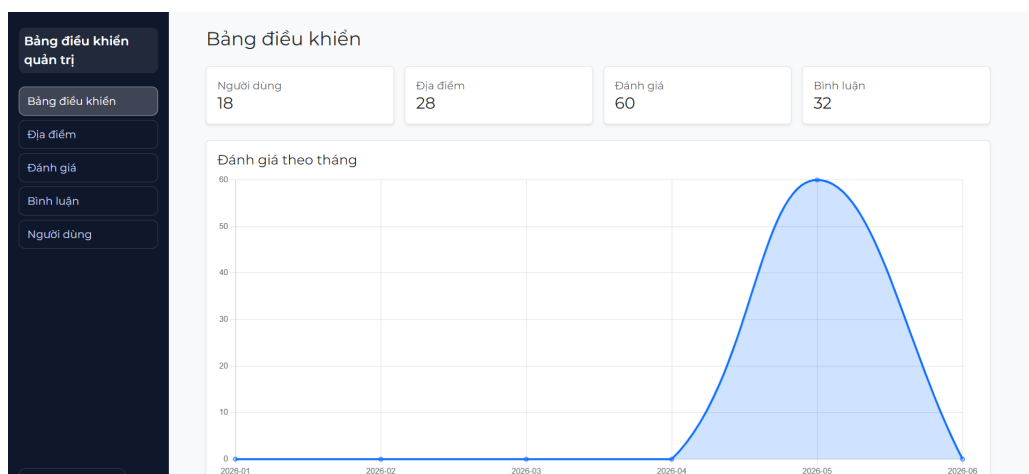


Hình 4.2: Giao diện Chi tiết điểm đến du lịch

Mô tả: Cung cấp đầy đủ thông tin hình ảnh, mô tả địa lý, bản đồ số, điểm số đánh giá trung bình và bộ lọc danh sách các bài review của người dùng.

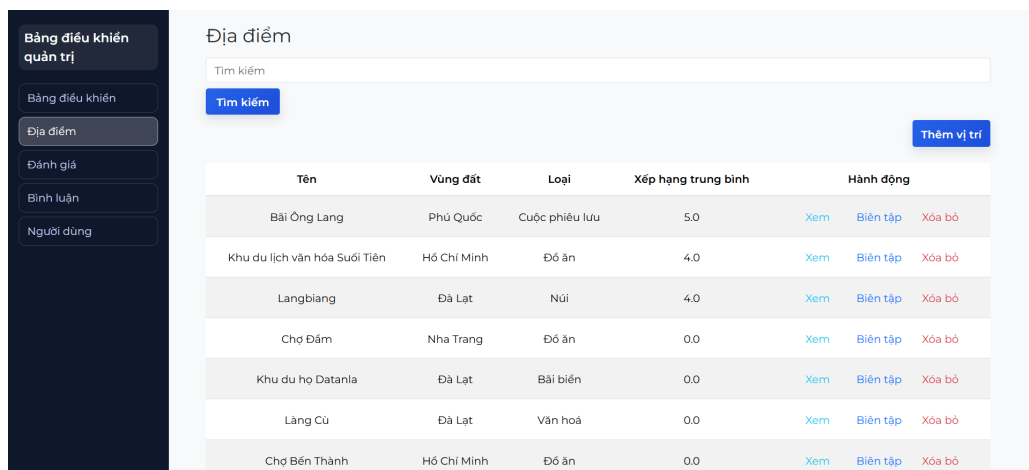
4.1.2 Giao diện phân hệ quản trị viên (Admin-side)

Phân hệ quản trị giúp người điều hành hệ thống dễ dàng giám sát số liệu và kiểm soát chất lượng nội dung một cách chặt chẽ.



Hình 4.3: Giao diện Trang quản trị hệ thống (Dashboard Admin)

Mô tả: Biểu đồ trực quan hóa dữ liệu thống kê lượng truy cập, số lượng tài khoản đăng ký mới, số bài review trong ngày và các chỉ số tăng trưởng.



Hình 4.4: Giao diện Chức năng quản lý và phê duyệt nội dung bài viết

Mô tả: Nơi quản trị viên thực hiện duyệt các bài viết mới hoặc xử lý các bài đánh giá bị cộng đồng báo cáo vi phạm tiêu chuẩn cộng đồng.

4.2 Chức năng đã hoàn thành

Đội ngũ phát triển đã hoàn thiện thành công các yêu cầu cốt lõi đặt ra ban đầu. Dưới đây là bảng tổng hợp tình trạng thực hiện các chức năng chính của hệ thống:

STT	Tên chức năng / Luồng nghiệp vụ	Trạng thái	Ghi chú kỹ thuật
1	Đăng ký, Đăng nhập & Phân quyền	Hoàn thành	Xác thực qua Token bảo mật, phân tách rõ quyền User và Admin.
2	Tìm kiếm & Lọc điểm đến thông minh	Hoàn thành	Tối ưu hóa truy vấn dữ liệu theo từ khóa và bộ lọc vị trí.
3	Tương tác (Tạo Review, Comment, Rate)	Hoàn thành	Luồng nghiệp vụ xử lý đồng bộ, không xảy ra xung đột database.
4	Thống kê & Tổng hợp số liệu (Dashboard)	Hoàn thành	Hiển thị số liệu thực tế qua các biểu đồ phân tích trực quan.
5	Kiểm duyệt nội dung & Quản lý bài đăng	Hoàn thành	Admin có toàn quyền ẩn/xóa dữ liệu vi phạm chuẩn mực.

Bảng 4.1: Bảng tổng hợp kết quả hoàn thiện chức năng hệ thống

Các kết quả đạt được cụ thể về mặt kỹ thuật bao gồm:

- **Độ ổn định cao trên môi trường máy chủ:** Hệ thống vận hành trơn tru, thời gian phản hồi (Response Time) ở mức thấp và các tiến trình nền được quản lý hiệu quả.
- **Tính toàn vẹn của dữ liệu:** Các luồng nghiệp vụ tương tác cốt lõi như viết Review, Comment hay chấm điểm (Rate) được xử lý thông qua cơ chế Transaction của hệ quản trị cơ sở dữ liệu, đảm bảo triệt để không xảy ra tình trạng trùng lặp hay sai lệch số liệu khi người dùng thao tác liên tục.

- **Hệ thống phân quyền nghiêm ngặt:** Đảm bảo an toàn thông tin bằng cách chặn hoàn toàn các truy cập trái phép từ phía client vào các endpoint hoặc API nội bộ dành riêng cho phân hệ Admin.

4.3 Kiểm thử và Đánh giá hệ thống

Để đảm bảo phần mềm vận hành đạt chuẩn và sẵn sàng đưa vào sử dụng, hệ thống đã được tiến hành thử nghiệm qua hai giai đoạn chính:

4.3.1 Kiểm thử chức năng (Functional Testing)

Đội ngũ đã xây dựng các kịch bản kiểm thử (Test cases) chi tiết cho các tính năng từ cơ bản đến nâng cao. Kết quả cho thấy toàn bộ các tính năng cốt lõi đều hoạt động đúng logic thiết kế và không ghi nhận lỗi hệ thống nghiêm trọng (Critical Bug).

4.3.2 Đánh giá khía cạnh an toàn thông tin

Hệ thống đã tích hợp các cơ chế bảo mật nền tảng bao gồm:

- Mã hóa một chiều mật khẩu người dùng trước khi lưu trữ vào cơ sở dữ liệu.
- Triển khai các bộ lọc kiểm tra dữ liệu đầu vào (Input Validation) nhằm phòng chống các hình thức tấn công mã độc phổ biến như SQL Injection và Cross-Site Scripting (XSS) tại các form nhập liệu công khai.

4.4 Hướng phát triển trong tương lai

Để hệ thống ngày càng hoàn thiện, đáp ứng tốt hơn nhu cầu thực tế của người dùng và nâng cao tính ứng dụng, một số hướng phát triển chiến lược trong tương lai bao gồm:

- **Chuẩn hóa hệ thống RESTful API:** Xây dựng và mở rộng hệ thống API độc lập, có tính bảo mật cao, tạo tiền đề để kết nối và phát triển đồng bộ ứng dụng trên thiết bị di động (Mobile App trên cả hai nền tảng iOS và Android).
- **Tích hợp Trí tuệ nhân tạo (AI):** Ứng dụng các mô hình xử lý ngôn ngữ tự nhiên (NLP) để phân tích sắc thái cảm xúc (Sentiment Analysis) của các bài review. Đồng thời, xây dựng bộ lọc thông minh tự động gắn cờ hoặc ẩn các bài viết có nội dung rác, spam quảng cáo hoặc xúc phạm thô tục.
- **Mở rộng module dịch vụ khép kín (Booking Ecosystem):** Tích hợp các dịch vụ của bên thứ ba cho phép người dùng có thể đặt tour du lịch trực tuyến, đặt phòng khách sạn hoặc mua vé tham quan trực tiếp ngay tại trang thông tin của địa điểm đang xem, tối ưu hóa hành trình trải nghiệm của khách hàng.

Phân chia công việc

Báo cáo bổ sung tự đánh giá của nhóm về thực tế đóng góp của các thành viên trong nhóm:

Lưu ý: Tổng đóng góp = 100%

STT	Họ và tên - MSSV	Nhiệm vụ đảm nhận	Đóng góp
1	Nguyễn Thế Anh 23016080	Phân tích & thiết kế CSDL (Migrations/Models). Xây dựng phân hệ Admin (Dashboard, kiểm duyệt Review/Comment, quản lý User).	34%
2	Nguyễn Mạnh Cường 23016117	Thiết kế giao diện (Blade views, CSS). Xây dựng chức năng Public (Trang chủ, chi tiết Location, tìm kiếm và lọc dữ liệu).	33%
3	Vũ Hùng Hiến 23016291	Xây dựng luồng nghiệp vụ người dùng (Auth, Rating, Favorite, viết Review). Tích hợp cấu trúc báo cáo.	33%

Tài liệu tham khảo

- [1] D. Buhalis, “Technology in tourism-from tourism technology to eTourism and smart tourism,” *Tourism Management Perspectives*, vol. 34, p. 100616, 2020.
- [2] B. A. Sparks and V. Browning, “The impact of online reviews on hotel booking intentions and perception of trust,” *Tourism Management*, vol. 32, no. 6, pp. 1310–1323, 2011.
- [3] T. Otwell, *Laravel 12.x - The PHP Framework for Web Artisans*, Laravel Official Documentation, 2025. [Online]. Available: <https://laravel.com/docs/>.
- [4] M. Zandstra, *PHP 8 Objects, Patterns, and Practice: Master Advanced Features and Design Patterns*, 6th Edition, Apress, 2021.
- [5] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th Edition, Pearson, 2015.
- [6] M. Edward, *Hypermedia and Performance Optimization in Modern PHP Object-Relational Mapping*, Academic Press, 2022.
- [7] OWASP, *Top 10 Web Application Security Risks*, Open Web Application Security Project, 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>.
- [8] Viện Nghiên cứu Phát triển Du lịch (ITDR), *Chuyển đổi số trong ngành Du lịch Việt Nam: Thực trạng và Giải pháp đột phá*, Nhà xuất bản Tổng cục Du lịch, Hà Nội, 2023.
- [9] W. Medhat, A. Hassan, and H. Korashy, “Sentiment analysis algorithms and applications: A survey,” *Ainu Shams Engineering Journal*, vol. 5, no. 4, pp. 1093–1113, 2014.
- [10] Nguyễn Văn Ba, *Phân tích và Thiết kế Hệ thống Thông tin theo hướng đối tượng với UML*, Nhà xuất bản Đại học Quốc gia Hà Nội, 2020.